《编译原理》

第11讲：词法分析(b)

何冬杰
重庆大学

《编译原理》

自动机理论相关内容补充

何冬杰
重庆大学

有穷自动机

- 有穷自动机只回答 Yes/No，分为两类
  - **不确定的有穷自动机** (Nondeterministic Finite Automata/NFA)：
    - 边上的标号没有限制
    - 一个符号可出现在离开同一个状态的多 条边上
    - ε可以做标号
  - **确定的有穷自动机** (Deterministic Finite Automata/DFA)：
    - 对于每个状态和每个输入符号，至多有一条出边
- 两种自动机都识别正则语言
  - 对于每个可以用正则表达式描述的语言， 均可用某个 NFA或DFA来识别；反之亦然
  - **识别**： 判定一个串是否属于对应正则语言(Yes/No)

不确定的有穷自动机

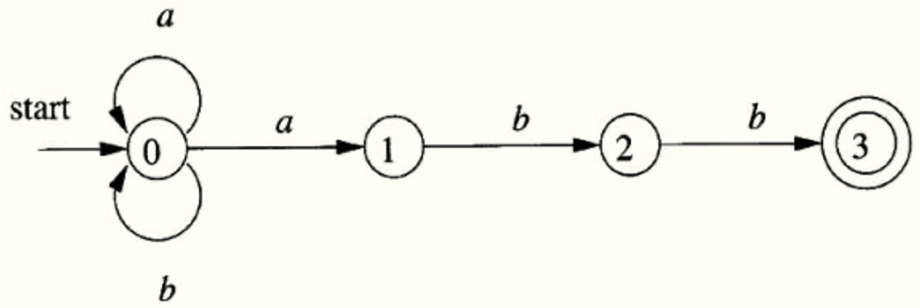
- NFA的定义：$\langle S, \sum, \phi, s_0, F \rangle$
  - 一个有穷的状态集合*S*
  - 一个输入符号字母表Σ (Input alphabet)
  - **转换函数** (Transition function) : ϕ: $S \rightarrow \sum \rightarrow \mathcal{P}(S)$
    - 对于每个状态和Σ ∪ {ε}中的符号， 给出相应的后继状态集合
  - *S*中的某个状态s_0 被指定为**开始状态/初始状态**(有些定义中可以有多个开始状态)
  - *S*的一个子集*F*被指定为**接受状态**集合

NFA的例子

- 状态集合 S = {0, 1, 2, 3}
- 输入字母表 Σ = {a, b}
- 开始状态 s_0 = 0
- 接受状态集合 F = {3}

δ(0, a) = {0, 1}　　δ(0, b) = {0}

δ(1, b) = {2}　　δ(2, b) = {3}



转换表 (Transition Table)

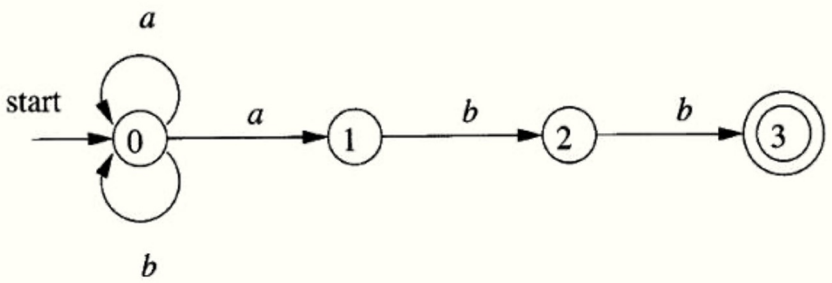
- 用二维表表示NFA的转换函数
  - 每行对应于一个状态
  - 每列对应于一个输入符号或者ε
  - 每个条目表示对应的后继状态集合

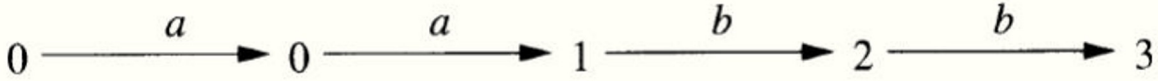
状态	a	b	ϵ
0	$\{0,1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

转换表表示法

输入字符串的接受

- 一个NFA接受输入字符串*x*
  - 当且仅当对应的转换图中存在一条从开始状态到某个接受状态的路径，且该路径各条边上的标号按顺序组成*x* (不含ε标号)
- 前面的NFA接受aabb，因为

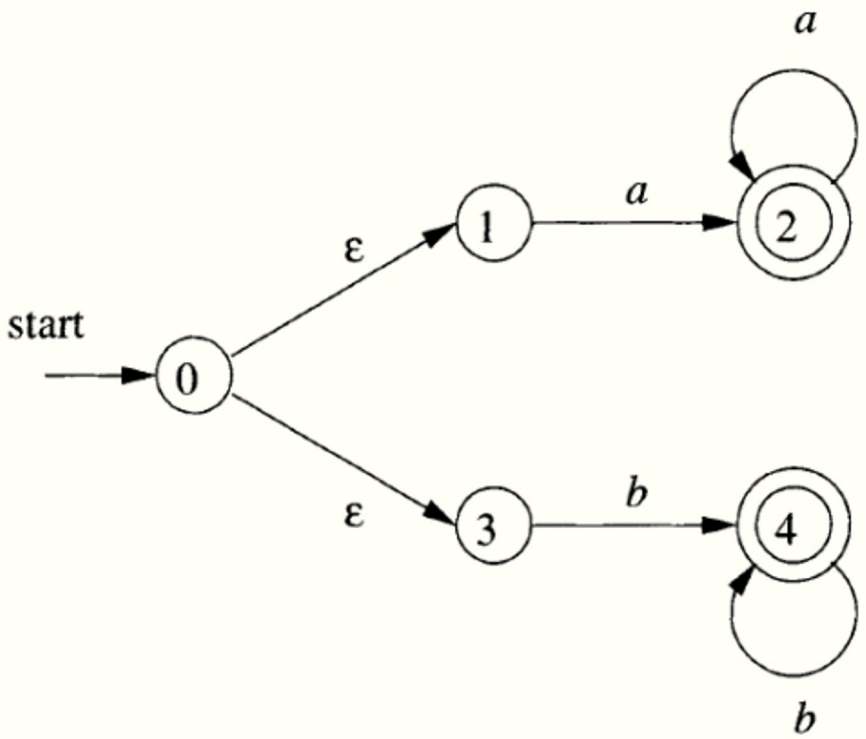




- **NFA接受的语言**：从开始状态到达接受状态的 所有可能路径的标号串的集合
  - 即该NFA接受的所有符号串的集合

NFA和相应语言的例子

- 相应的语言： L(aa* | bb*)



从正则表达式到自动机的转换

- 正则表达式： 简洁、精确地描述词法单元的模式
- 自动机： 便于机器执行的计算模型
  - NFA：易于从正则表达式转换(ε跳转)
  - DFA：可由NFA转换， 便于机器高效模拟执行
  - DFA最小化，优化空间使用率

正则表达式 → NFA → DFA

正则表达式到NFA（Thompson构造法）

❑基本思想

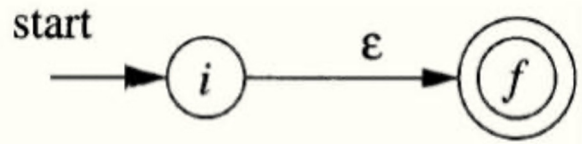
- 根据正则表达式的递归定义， 按照正则表达式的结构递归地构造出相应的NFA
- 算法分成两个部分
  - 基本规则处理ε和单符号的情况
  - 对于每个正则表达式的运算，建立组合相应NFA的方法

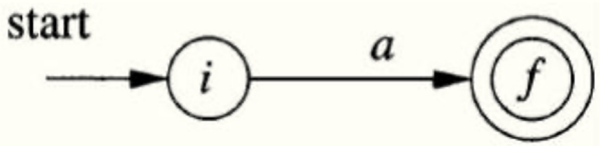
∅ (matches no string)	Concatenation $(R_1\ R_2)$
ε (epsilon – matches empty string)	Alternation $(R_1 \mid R_2)$
Literals (‘a’, ‘b’, ‘2’, ‘+’, etc.) drawn from alphabet	Kleene star $(R\ *)$

正则表达式文法

转换算法

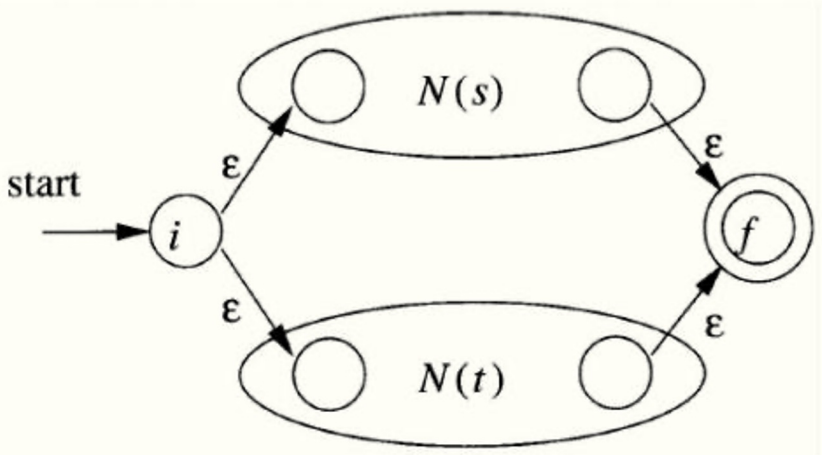
- 基本规则部分
  - 表达式ε
  - 表达式*a*



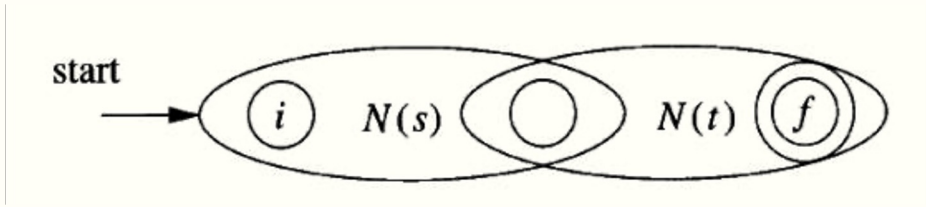


转换算法

- 归纳部分
  - $s \mid t$



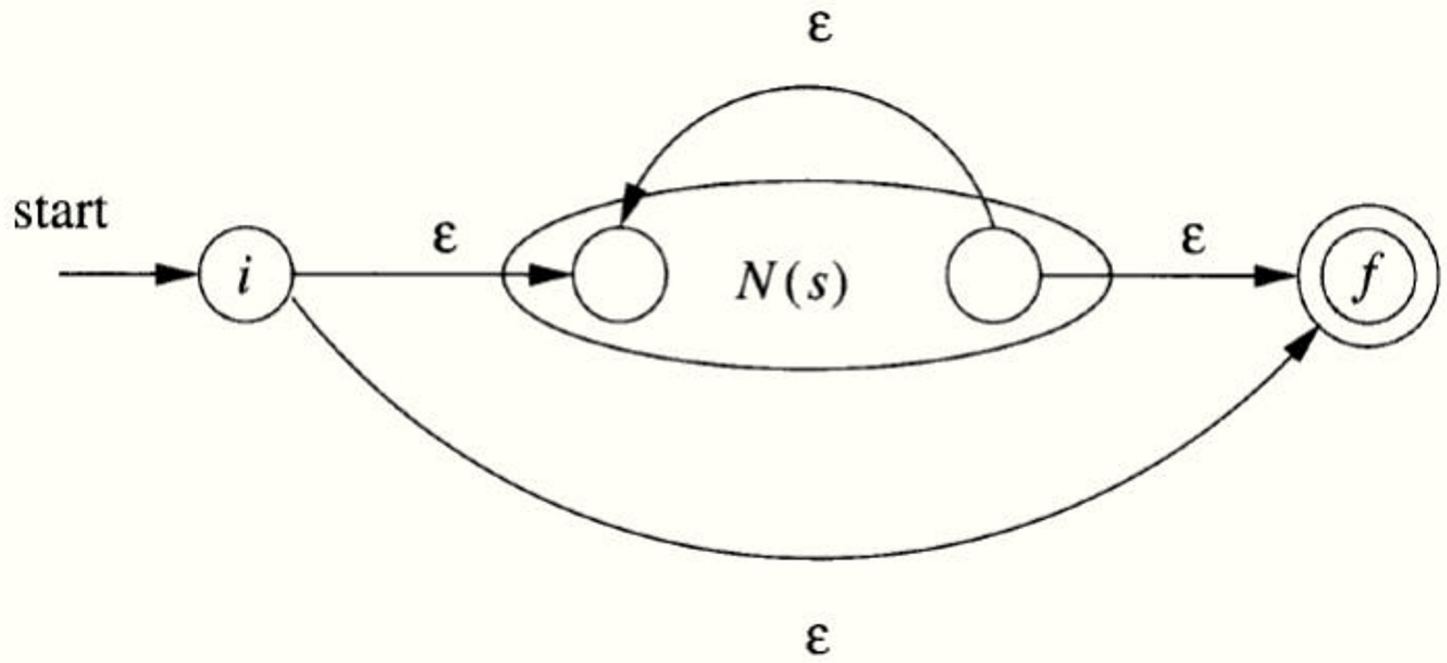
  - st



N(s)：识别正则表达式 s 的 NFA

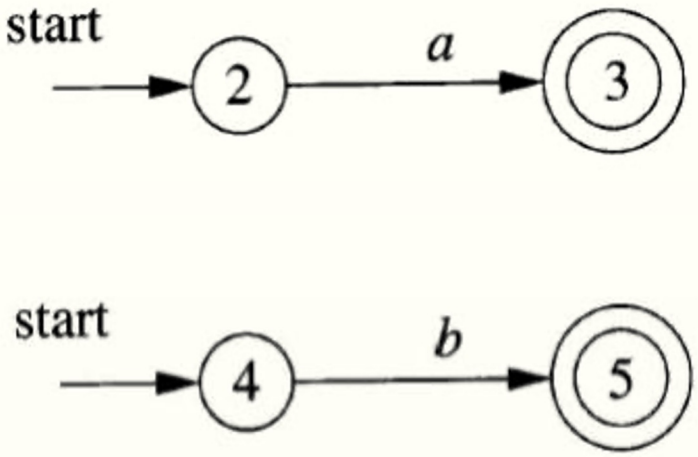
转换算法

- 归纳部分
  - s^* = { ε, s, ss, sss, $ssss$, ... }



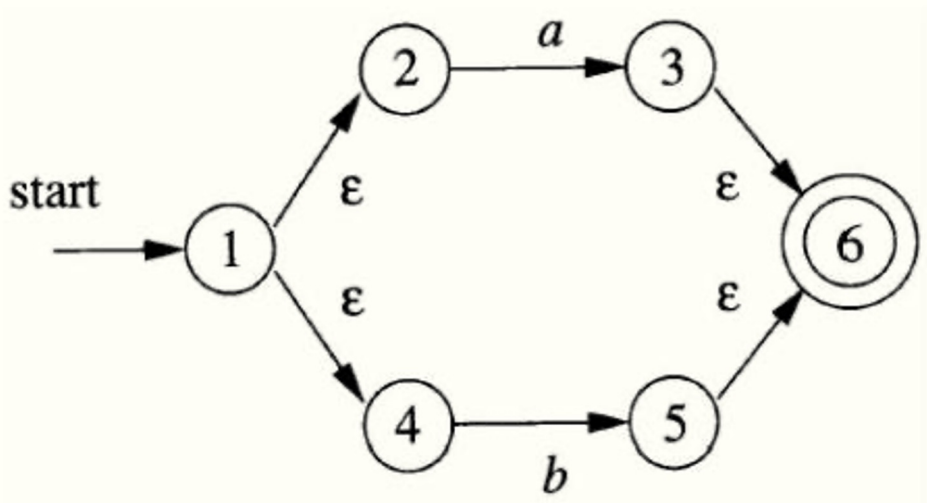
正则表达式到NFA的例子

- 正则表达式： (a|b)*abb
  - 第一个a对应的NFA
  - 第一个b对应的NFA



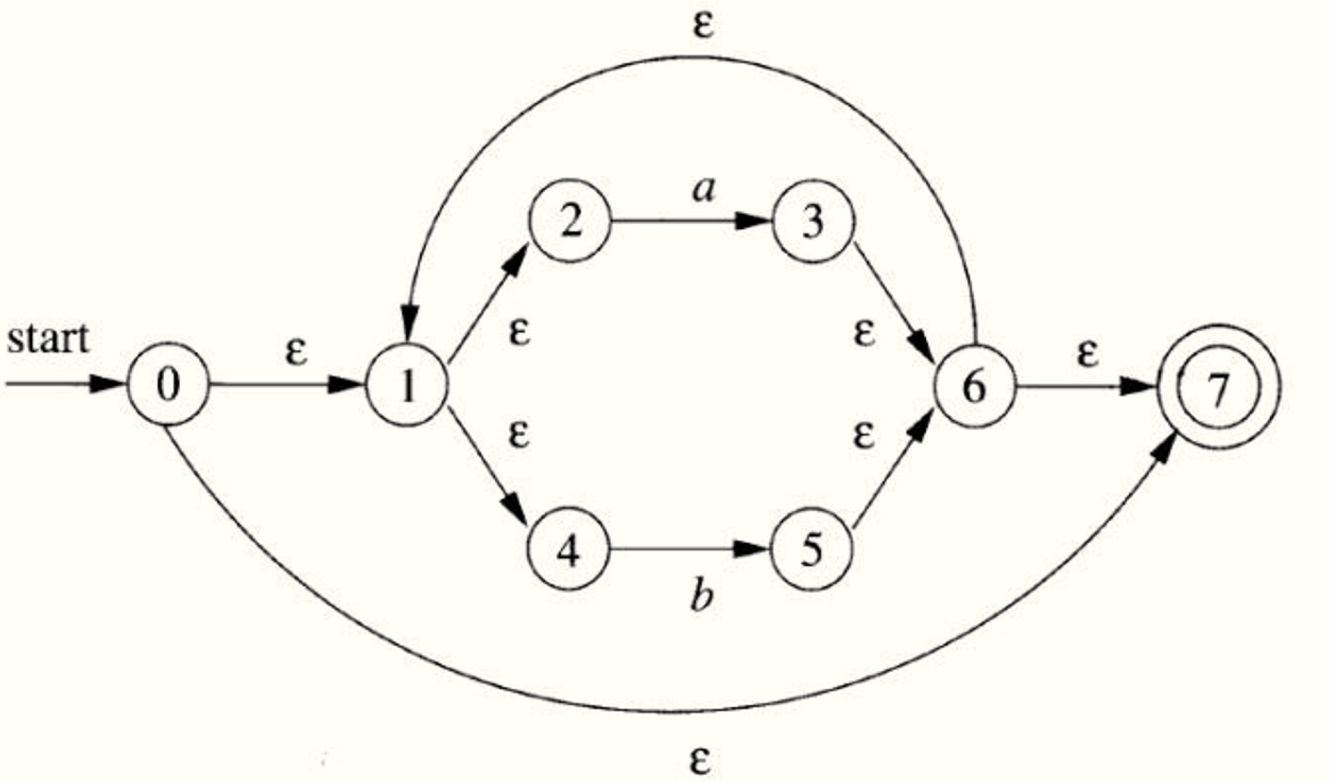
正则表达式到NFA的例子

- (a|b)*abb的NFA
  - a|b的NFA



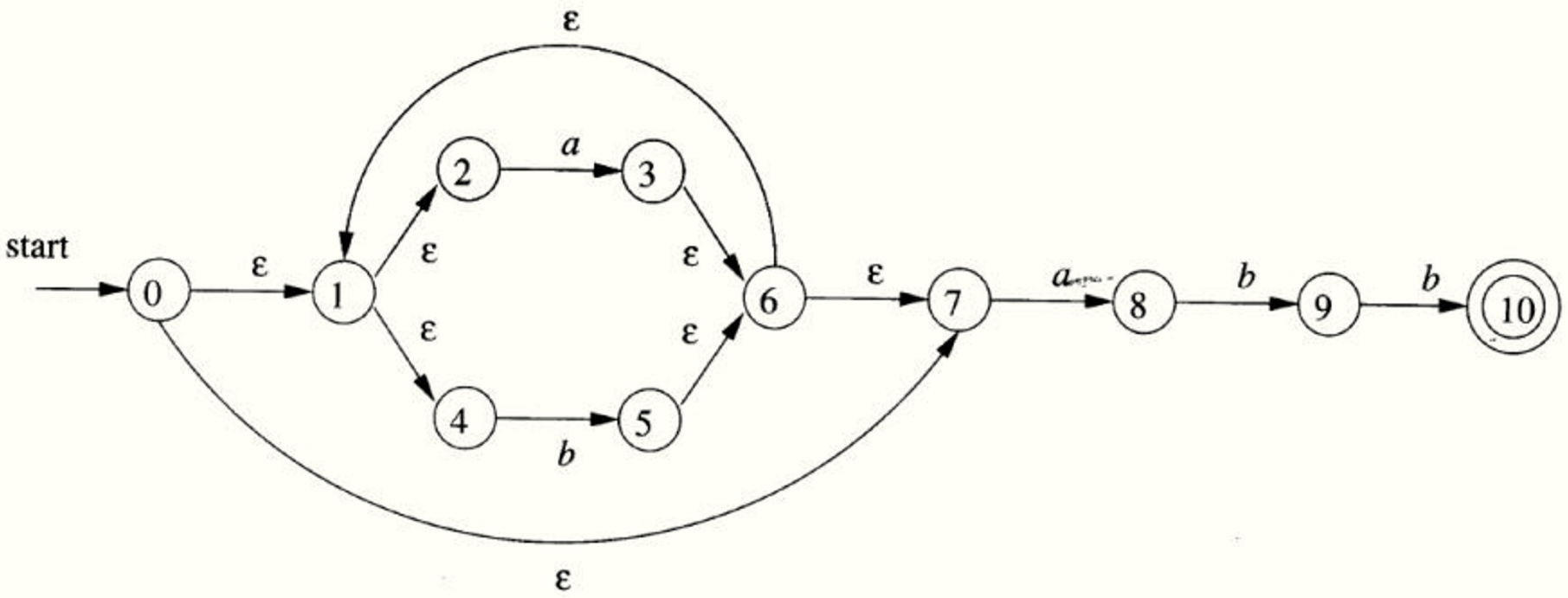
正则表达式到NFA的例子

- (a|b)*abb的NFA
  - **(a|b)*的NFA**



正则表达式到NFA的例子

- (a|b)*abb的NFA



确定的有穷自动机 (DFA)

- NFA 的不确定性使其不易模拟
  - 需要维护多个状态，跟踪多条路径
- DFA ($\langle S, \sum, s_0, \phi: S \to \sum \to S, F \rangle$)满足以下条件：
  - 没有标号为 ε 的转换
  - 每个状态对每个输入符号至多有一条出边
- DFA 可以高效判断一个串是否被接受

- **每个 NFA 都有一个接受相同语言的等价 DFA**

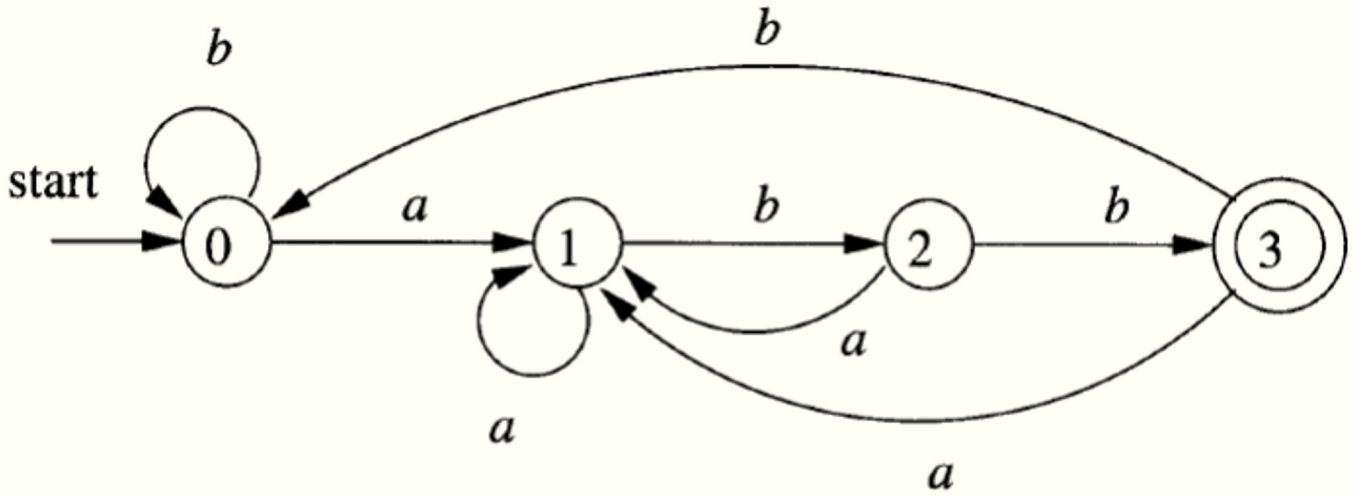
DFA的模拟运行

- 假设输入符号是字符
- *nextChar*读入下一个字符
- *move*给出了离开状态*s*且标号为*c*的边的目标状态

```
s = s0;
c = nextChar();
while ( c != eof ) {
        s = move(s, c);
        c = nextChar();
}
if ( s在F中 ) return "yes";
else return "no";
```

DFA的例子

- 假设输入为ababb，那么进入的状态序列为
  - 0, 1, 2, 1, 2, 3，返回yes



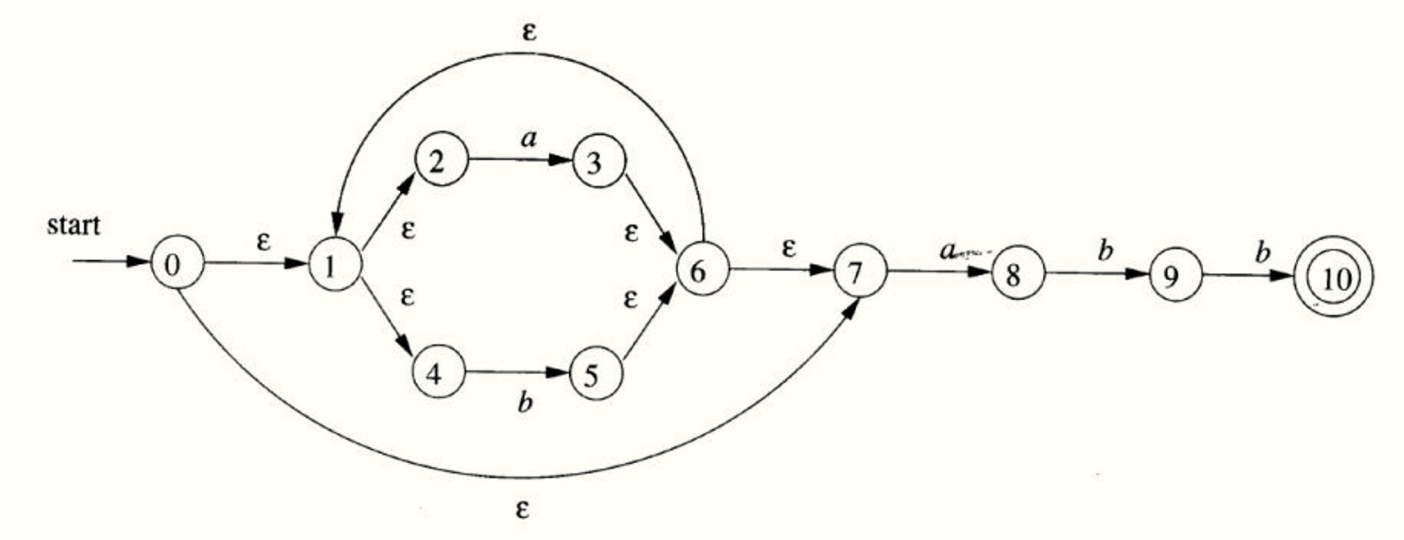
接受(a|b)*abb的DFA

NFA到DFA (子集构造法)

- 基本思想
  - 构造得到的DFA的每个状态和NFA的状态子集对应
  - DFA读入$a_1, a_2, \ldots, a_n$后到达的状态对应于从NFA开始状态出发沿着$a_1, a_2, \ldots, a_n$可能到达的状态集合
  - 在算法中“**并行地模拟**”NFA在遇到一个给定输入串时可能执行的所有动作

NFA例子

- 下面的NFA能够接受串babb
  - 考虑从开始状态出发， 沿着标号分别为b，ba，bab， babb能到达的所有可能状态的集合



起始：{0,1,2,4,7}

b：{1,2,4,5,6,7}

ba：{1,2,3,4,6,7,8}

bab：{1,2,4,5,6,7,9}

babb：{1,2,4,5,6,7,10}

模拟 NFA 的状态转换：先沿输入符号转换，再求 ε 闭包

NFA到DFA (子集构造法)

- 目标DFA每个状态对应一个NFA状态集合（子集）
- 目标DFA状态之间的转换对应NFA状态集合之间的转换
- 理论上，最坏情况下DFA的状态个数会是NFA状态个数的指数多个（幂集，2^n）
  - 但对于大部分应用，NFA和相应的DFA的状态数量大致相同

NFA到DFA(子集构造法)

- 算法中使用到的基本操作
  - move(*T*, *a*)：枚举*T*中所有状态，从*T*中某个状态*s*出发，通过一个标号为*a*的转换能到达的NFA状态集合
  - ε–closure(*s*)：从NFA状态*s*开始，只通过ε转换能到达的NFA状态集合
  - ε–closure(*T*)：枚举*T*中所有状态，从*T*中某个状态*s*开始，只通过ε转换能到达的NFA状态集合

标号转换 + ε 转换

NFA到DFA (子集构造法)

- 计算ε–closure(*T*)的算法
  - 实际上是一个图搜索过程 (只考虑ε标号边)

```
将T的所有状态压入stack中；
将 ε-closure(T) 初始化为T;
while ( stack非空) {
        将栈顶元素t弹出栈中;
        for (每个满足如下条件的u:从t出发有一个标号为ε的转换到达状态u)
                if ( u不在 ε-closure(T)中) {
                        将u加入到ε-closure(T)中;
                        将u压入栈中;
                }
}
```

NFA到DFA(子集构造法)

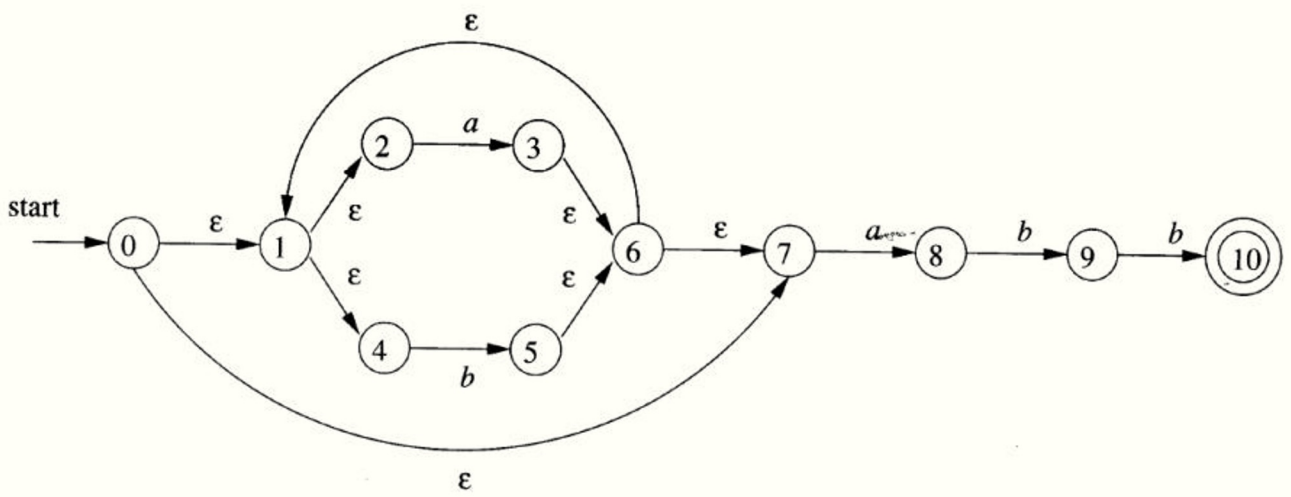
- 算法输出
  - *Dstates*: DFA状态， 每个元素对应一个NFA状态集合
  - *Dtran:* DFA状态转换表

```
一开始, ε-closure(s0)是 Dstates 中的唯一状态, 且它未加标记;
while (在 Dstates 中有一个未标记状态 T ) {
    给T加上标记;
    for ( 每个输入符号 a ) {
        U = ε-closure(move(T, a));
        if ( U 不在Dstates中 )
            将 U 加入到Dstates中, 且不加标记;
        Dtran[T, a] = U;
    }
}
```

加标记 = visited

子集构造法的例子

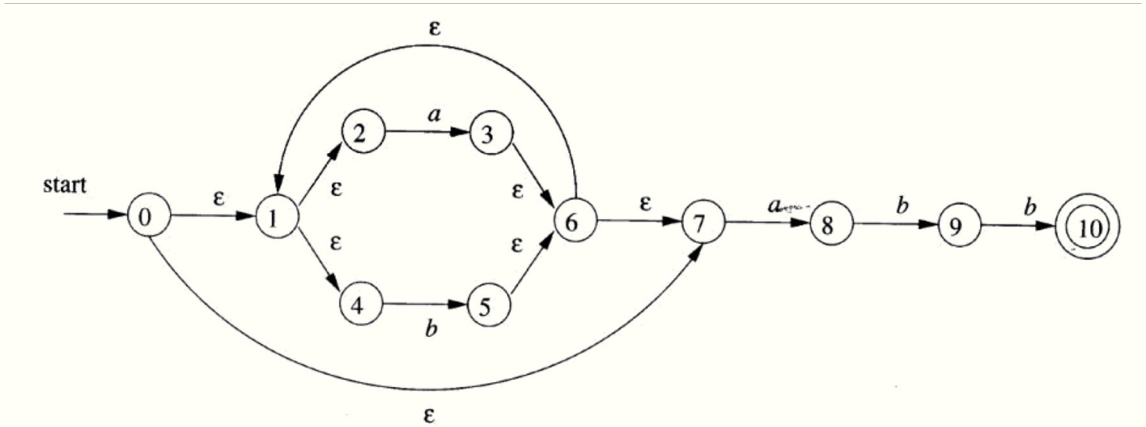
- A: ε–closure(0) = {0, 1, 2, 4, 7}
- B: Dtran[A, a] = ε–closure({3, 8}) = {1, 2, 3, 4, 6, 7, 8}
- C: Dtran[A, b] = ε–closure({5}) = {1, 2, 4, 5, 6, 7}
- D: Dtran[B, b] = ε–closure({5, 9}) = {1, 2, 4, 5, 6, 7, 9}
- …



子集构造法的例子

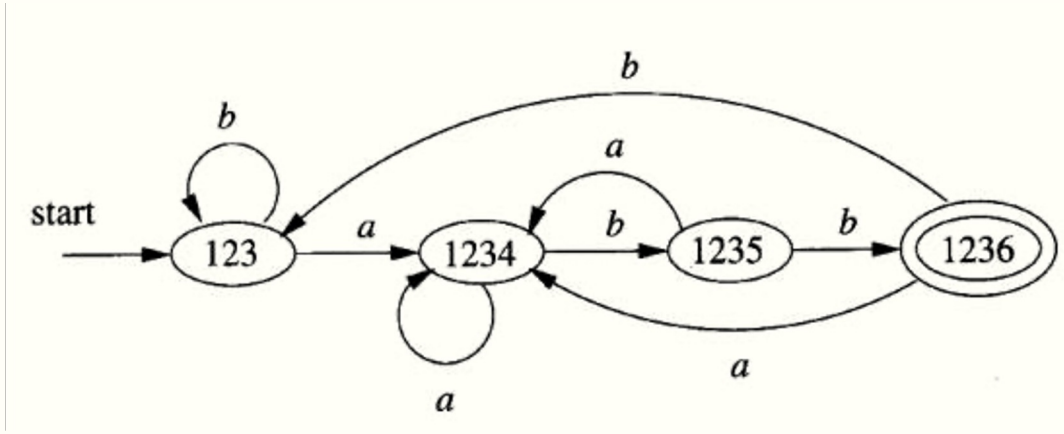
- 开始状态： A；接受状态： E

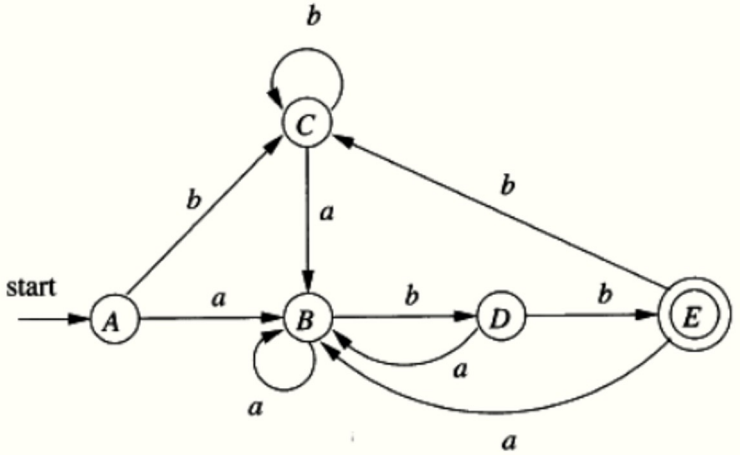
NFA 状态	DFA 状态	a	b
$\{0, 1, 2, 4, 7\}$	A	B	C
$\{1, 2, 3, 4, 6, 7, 8\}$	B	B	D
$\{1, 2, 4, 5, 6, 7\}$	C	B	C
$\{1, 2, 4, 5, 6, 7, 9\}$	D	B	E
$\{1, 2, 4, 5, 6, 7, 10\}$	E	B	C



DFA状态数量的最小化

- DFA状态越少， 空间效率越高
- 一个正则语言可对应于多个识别此语言的DFA
- 通过**DFA的最小化**可得到状态数量最少的DFA (不计同构， 这样的DFA是唯一的)

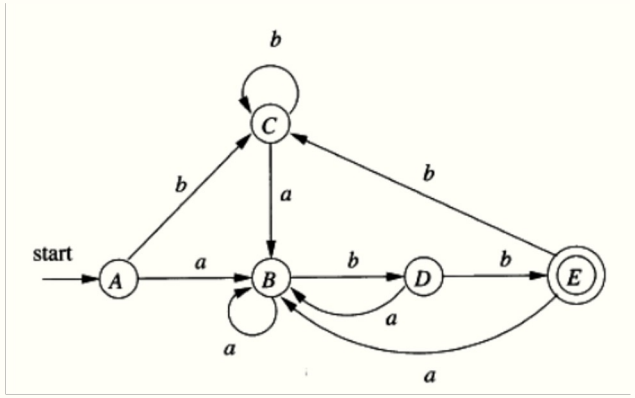




两个等价的 DFA 都识别 (a|b)*abb

状态的区分

- 状态的**可区分**
  - 如果存在串*x*，使得从状态s1和s2，一个到达接受状 态而另一个到达非接受状态，那么*x*就区分了s1和s2
  - 如果存在某个串区分了*s*和*t*，我们说*s*和*t*是**可区分**的，否则它们是**不可区分**的
- 不可区分的两个状态就是**等价**的，可以合并



空串区分了E和其它状态； bb区分了A和B

DFA最小化算法

- 把所有可区分的状态分开 (迭代过程)
  - 基本步骤： ε区分了接受状态和非接受状态
  - 归纳步骤： 如果状态*s*和*t*是可区分的， 且*s*' 到*s*、 *t*' 到*t*有标号为a的边，那么*s*'和*t*'也是可区分的
- 最终没有区分开的状态就是等价的
  - 所有的**死状态**都是等价的
- 从划分得到的等价类中选取代表，并重建DFA

最小化算法 (划分部分)

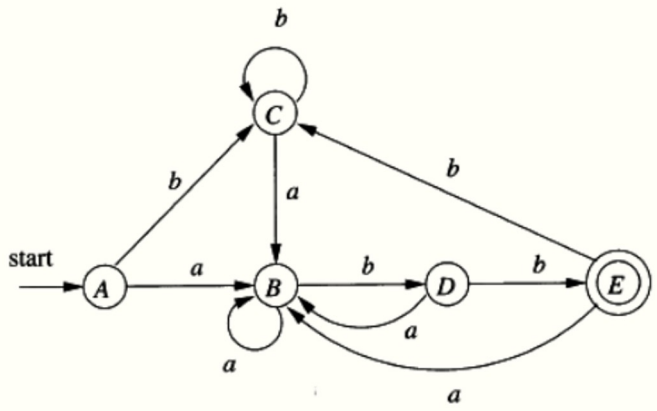
- 设置初始划分： $\Pi = \{ S _ F, F \}$
- 迭代， 枚举字母表， 不断划分等价类
  - for (Π中的每个元素G) {
  - 细分G，使得G中的s、 t仍然在同一组中iff
  - 对任意输入a，s、 t都到达Π中的同一组；
  - Π_{new} = 将Π中的G替换为细分得到的小组 }
- 如果$\Pi_{new} == \Pi$,令$\Pi_{final} = \Pi$,算法完成； 否则$\Pi = \Pi_{new}$， 转步骤2

最小化算法 (构造部分)

- 在Π_{final}的每个组中选择一个状态作代表，作为最小化DFA中的状态
  - **开始状态**就是包含原开始状态的组的代表
  - **接受状态**就是包含了原接受状态的组的代表(这个组一定只包含接受状态)
  - 转换关系构造如下
    - 如果*s*是*G*的代表，而原DFA中*s*在*a*上的转换到达*t*，且*t*所在组的代表为*r*，那么最小化DFA中有从*s*到*r*的在*a*上的转换

DFA最小化的例子

- 初始划分：{{A, B, C, D}, {E}}
- b 将 {A, B, C, D} 分成 {A, B, C} 和 {D}
  - 得到 {{A, B, C}, {D}, {E}}
- b 将 {A, B, C} 分成 {A, C} 和 {B}
  - 得到 {{A, C}, {B}, {D}, {E}}
- 选取 A、B、D、E 为代表，构造最小 DFA

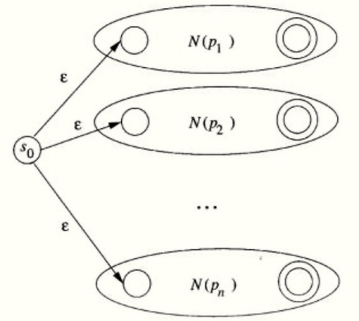


状态	a	b
A	B	A
B	B	D
D	B	E
E	B	A

图 3-65　状态最少 DFA 的转换表

自动机到词法分析器

- 正则表达式： 识别单个模式
- 词法分析器： 识别多种模式
  - 关键字、 运算符、 标识符、 数字
- NFA合并方法
  - 引入新的开始状态，并引入从该开始状态到各个原开始状态的ε转换
  - 得到的NFA所接受的语言是原来各个NFA语言的并集
  - 不同的接受状态代表不同的模式



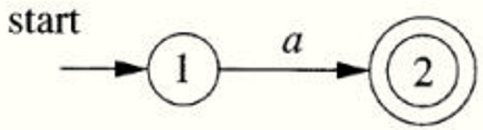
确定化NFA可能引发冲突

- 对得到的NFA进行确定化，得到DFA
- 一个DFA的接受状态对应于NFA的状态子集， 其中至少包括一个NFA的接受状态
  - 如果其中包括多个对应于不同模式的NFA接受状态，则表示**当前的输入前缀对应于多个模式**， 存在冲突
  - 找出第一个这样的模式， 将该模式作为此DFA接受状态的输出

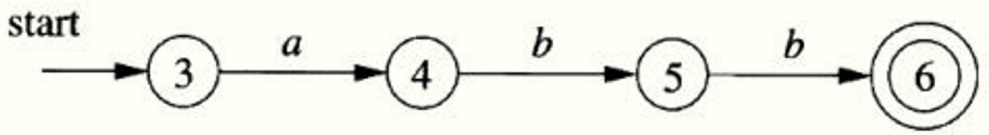
例子

- 假设有三个模式, 构造各模式的NFA

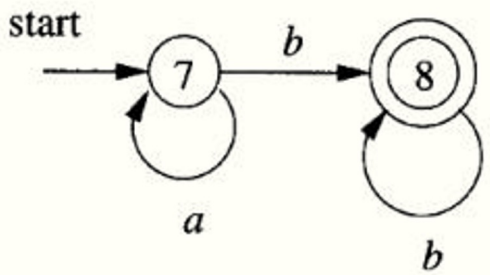
- 模式1：a



- 模式2：abb

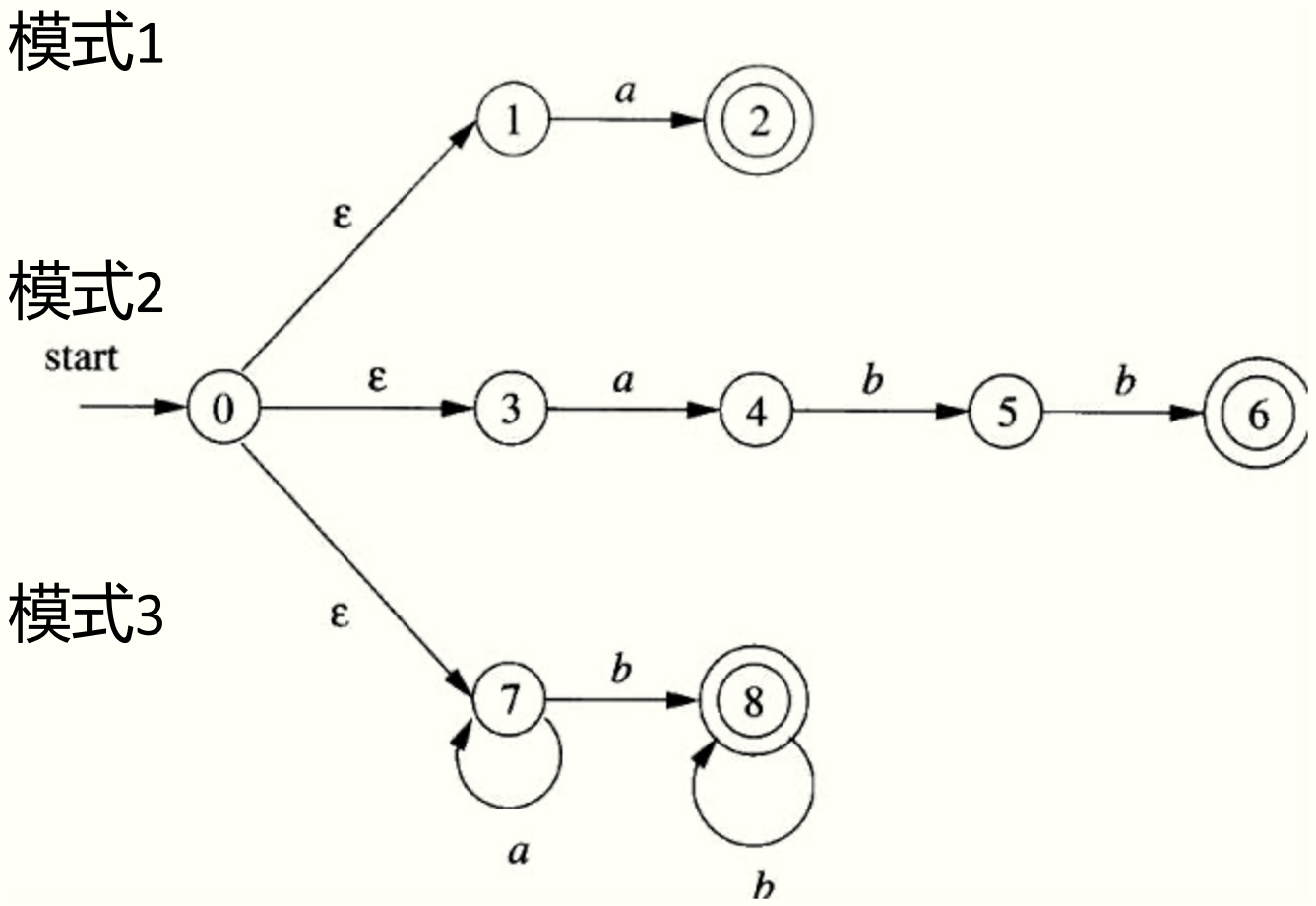


- 模式3：a^*b^+



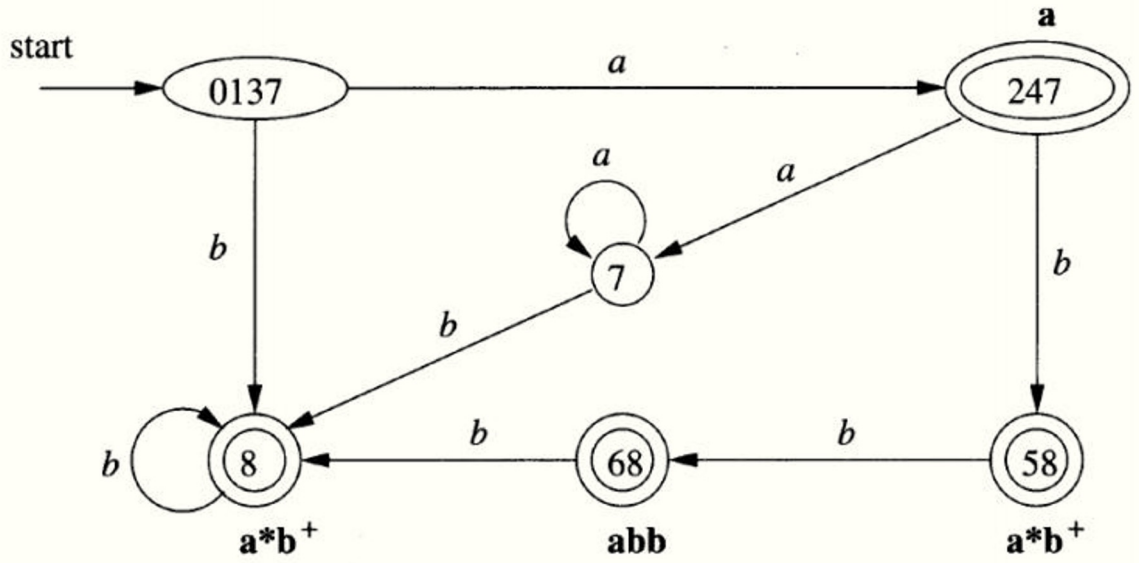
例子

- 合并NFA
  - 2：模式1
  - 6：模式2
  - 8：模式3



词法规则的冲突与优先级

- 合并 NFA 后，确定化得到 DFA
- 状态 68 同时匹配两种模式

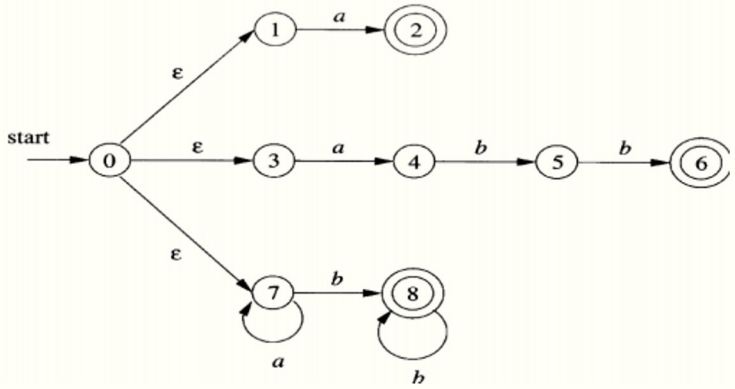


模式 3　　模式 2　　模式 3

模式1

模式2

模式3



状态 68 = {6, 8}

模式 2： abb

模式 3： a*b+

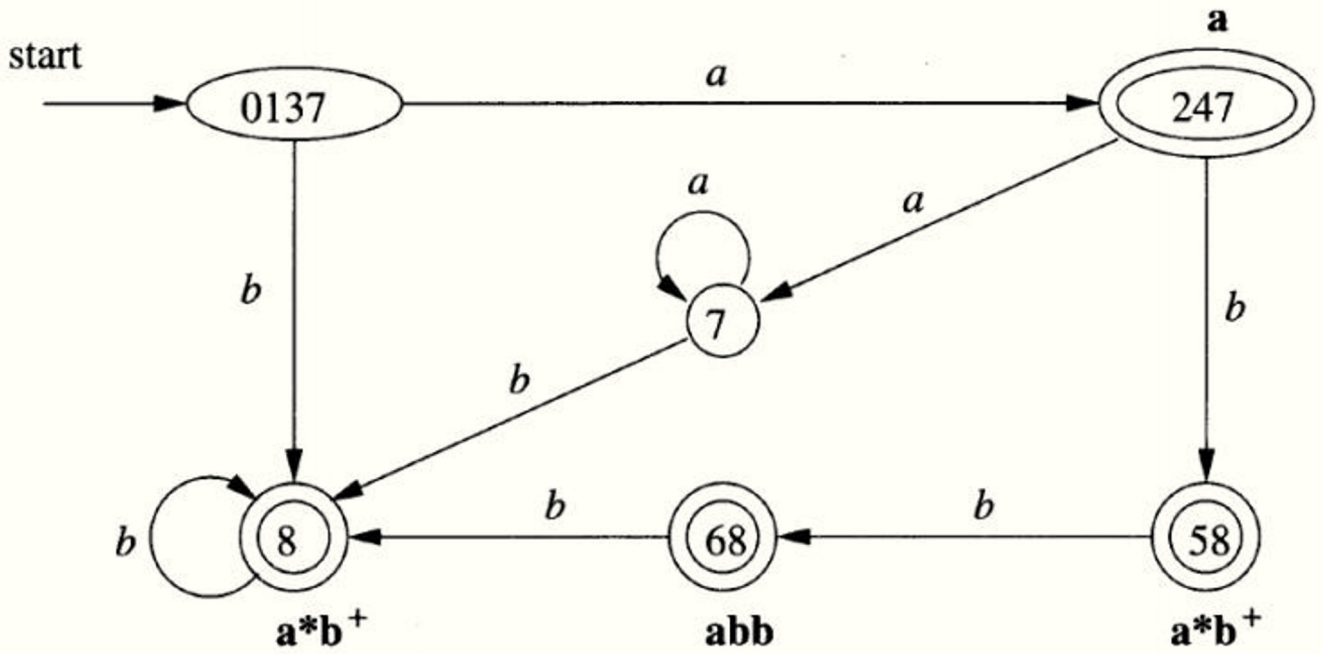
按规则优先级选择模式 2

词法分析器状态的最小化

- 基本思想和DFA最小化算法相同
- 差别
  - 词法分析器中的接受状态对应于不同的模式
  - 对应不同模式的接受状态一定是不等价的
  - 初始划分为
    - 所有非接受状态集合 + 对应各模式的接受状态集合
- 其余划分和构造的方法均相同
- 接受状态对应的模式就是原来的模式

词法分析器最小化的例子

- 初始划分： {0137, 7}, {247}, {68}, {8, 58}, {Φ}
  - 增加死状态Φ



模式1

模式 3　　模式 2　　模式 3

作业: RE->NFA->DFA->DFA_{min}练习

- 设字母表$\sum = \{a, b\}$, 正则表达式$R = a(a|b)^*b$可以识别以a开头，以b结尾的字符串

- 练习：
  - 使用Thompson构造法将该正则表达式转换为NFA
  - 使用子集构造法将得到的NFA转为完整的DFA，给出状态转换表
  - 使用划分细化法最小化该DFA，并画出最小DFA
  - 在最小DFA上追踪`ab`, `aba`, `aabb`和`bab`的路径，判断是否接受